

บทที่ 4

การสร้างชิ้นงาน

ในบทนี้ จะกล่าวถึงการสร้างชิ้นงานหรือการเขียนโปรแกรมทั้งหมดที่ต้องใช้ในระบบ เนื่องจากมาตรฐาน OSGI ได้ออกแบบให้เฟรมเวิร์กทำงานอยู่บน JVM (Java Virtual Machine) และรองรับการพัฒนาเซอร์วิสบนเกตเวย์ด้วยจาวา ดังนั้นโปรแกรมทั้งหมดในโครงการนี้จึงพัฒนาด้วยภาษาจาวาใช้ฐานข้อมูล mySQL เนื่องจากเป็น open source และง่ายต่อการใช้งาน

ในการพัฒนาระบบจะเป็นการพัฒนาบน Microsoft Windows XP โปรแกรมทั้งหมดที่ต้องติดตั้งเพื่อใช้ในการสร้างระบบมีดังนี้

- | | |
|----------------------------------|--|
| 1. Apache Tomcat 5.0 | ใช้เป็นเว็บเซิร์ฟเวอร์ |
| 2. J2SE sdk 1.4.2 | ใช้ในการพัฒนาโปรแกรมในภาษาจาวา |
| 3. Knopflerfish 1.3.4 | ใช้เป็นโปรแกรมจำลองเกตเวย์ |
| 4. MySQL 5.0 | ใช้เป็นฐานข้อมูล |
| 5. MySQL Connector (JDBC driver) | ใช้เป็นตัวกลางเชื่อมต่อกับฐานข้อมูล mySQL ในภาษาจาวา |
| 6. EditPlus 2 | ใช้เป็นเอดิเตอร์ |
| 7. Internet Explorer | ใช้เป็นเว็บเบราว์เซอร์ |

การพัฒนาระบบมีองค์ประกอบดังต่อไปนี้

4.1 เรสซิเดนเชียลเกตเวย์

การในจำลองเกตเวย์นั้นได้เลือกใช้โปรแกรม Knopflerfish ในการจำลองเนื่องจากง่ายต่อการใช้งานรองรับมาตรฐาน OSGI ได้ 100% ทำให้มั่นใจได้ว่าเซอร์วิสที่พัฒนาขึ้นนี้สามารถนำไปใช้จริงได้ในอนาคต

4.1.1 ส่วนของโปรโตคอลเครือข่ายภายในบ้าน MBus

ส่วนของโปรโตคอลเครือข่ายภายในบ้าน MBus ประกอบด้วยการจำลองพอร์ตรับการเชื่อมต่อจากอุปกรณ์ภายในบ้าน เซอร์วิส “DriverLocator” ซึ่งทำหน้าที่หาและติดตั้งไดรฟ์เวอร์ของอุปกรณ์ที่มาเชื่อมต่อ และไดรฟ์เวอร์ของ MBus ซึ่งทำหน้าที่ส่งอ็อบเจกต์ระหว่างองค์ประกอบต่างๆ ในเครือข่าย Home-Connected เป็นผู้รับผิดชอบการออกแบบและพัฒนา Mbus

เนื่องจากองค์ประกอบต่างๆ ในเครือข่ายต้องอาศัย Mbus ในการติดต่อระหว่างกัน ดังนั้นจึงได้ทำการพัฒนาส่วนของ MBus ขึ้นก่อนองค์ประกอบอื่น

Mbus เป็นบันเดิลที่ประกอบด้วยเซอร์วิสที่มีลักษณะ “thread enabled” รอรับฟังการเชื่อมต่อที่พอร์ต 4413 ถ้ามีอุปกรณ์เชื่อมต่อเข้ามา MBus จะเรียกใช้เซอร์วิส DriverLocator ให้หาไดรฟ์เวอร์ของอุปกรณ์นั้นๆ

4.1.1.1 ไดรฟ์เวอร์ของ MBus

เป็นบันเดิลที่บรรจุคลาสชื่อ Activator ซึ่งจะเปิดพอร์ตที่ 4413 รอฟังการเชื่อมต่อเมื่อบันเดิลเริ่มทำงาน และทำการลงทะเบียนเซอร์วิส DriverLocator

เมื่อได้รับการเชื่อมต่อจากอุปกรณ์ MBus จะแตก thread ไปดูแลและสร้างอ็อบเจกต์ของคลาส MBusServiceImpl มาใช้ในแยกแยะและติดต่อกับอุปกรณ์ และเรียกใช้เซอร์วิส DriverLocator เมื่อทำการแยกแยะอุปกรณ์ได้แล้ว MBus จะกลับไปรอฟังการเชื่อมต่อจากอุปกรณ์ตัวใหม่ต่อไป

ผู้ให้บริการเซอร์วิสต่างๆต้องพัฒนาเซอร์วิสให้รองรับการใช้ message ของ MBus ในการติดต่อกัน ซึ่งประกอบด้วย Message ดังต่อไปนี้

4.1.1.1.1 MBusMessage

MBusMessage เป็นคลาส message พื้นฐานให้ message อื่นสืบทอดไปใช้ คลาส MBusMessage จะมีฟังก์ชันในการคืนค่าชนิด message (อาจเป็นชนิด MBusMessage.MBusID ซึ่งใช้บรรจุ ID ของอุปกรณ์ หรือเป็นชนิด MBusMessage.DeviceSpecific ซึ่งบรรจุข้อมูลที่ส่งให้แก่ละเซอร์วิสในเครือข่าย)

เพื่อให้ทำงานในสภาพแวดล้อมแบบไคลเอนต์-เซิร์ฟเวอร์ได้ MBusMessage จะบรรจุฟังก์ชันที่ใช้ในการแท็ก(tag) message เพื่อใช้ในการจับคู่การร้องขอ และการตอบกลับที่เป็นคู่กัน

4.1.1.1.2 MBusID

MBusID เป็นคลาสที่สืบทอดคุณสมบัติมาจากคลาส MBusMessage และเพิ่มฟังก์ชันใหม่คือฟังก์ชัน getID() ทำหน้าที่คืนค่า ID ของอุปกรณ์ซึ่งจะถูกส่งไปให้เซอร์วิส DriverLocator เพื่อใช้ในการหาไดรฟ์เวอร์มาติดตั้งโดยอัตโนมัติ

4.1.1.1.3 MBusDeviceSpecific

MBusDeviceSpecific เป็นคลาสที่สืบทอดคุณสมบัติมาจากคลาส MBusMessage ซึ่งใช้บรรจุข้อมูลส่งให้เซอร์วิสต่างๆในเครือข่าย ไดรฟ์เวอร์ของ MBus จะไม่รู้จักข้อมูลภายในอ็อบเจกต์นี้เห็นเป็นเพียงไบต์ของข้อมูล และส่งไปยังเซอร์วิสปลายทาง ซึ่งจะเป็นหน้าที่ของเซอร์วิสปลายทางในการแปลงไบต์กลับไปเป็นข้อมูลดั้งเดิมที่นำไปใช้ได้ โดยอาศัยแพ็คเกจ Java Base64 utilities ในการแปลง

4.1.1.2 เซอร์วิส DriverLocator

ในเฟรมเวิร์คอาจมีเซอร์วิส DriverLocator มากกว่า 1 ตัวก็ได้ เมื่อมีการตรวจพบอุปกรณ์ใหม่ในระบบ เฟรมเวิร์คของ OSGI จะสอบถามไปยังเซอร์วิส DriverLocator ทุกตัวว่าสามารถแยกแยะและหาไดรฟ์เวอร์ของอุปกรณ์ได้หรือไม่

เนื่องจากเราไม่คาดหวังว่าเซอร์วิส DriverLocator ตัวอื่นจะสามารถแยกแยะและหาไดรฟ์เวอร์ของอุปกรณ์ที่ใช้โปรโตคอล MBus ได้ ดังนั้นไดรฟ์เวอร์ของ MBus จึงลงทะเบียนเซอร์วิส DriverLocator ของตนเอง

เซอร์วิส DriverLocator หนึ่งๆจะเป็นไฟล์ .class ที่ใช้งานอินเตอร์เฟส DriverLocator ของเฟรมเวิร์ค OSGI ซึ่งประกอบด้วย 2 ฟังก์ชันคือ findDrivers และ loadDriver

ฟังก์ชัน findDrivers จะตรวจสอบตัวแปร DEVICE_CATEGORY DEVICE_MAKE และ MBUS_ID ของอุปกรณ์ที่เชื่อมต่อเข้ามาเพื่อให้แน่ใจว่าเป็นอุปกรณ์ที่รองรับโปรโตคอล MBus และตรวจสอบว่าสามารถดาวน์โหลดไดรฟ์เวอร์ได้หรือไม่ ตัวแปร MBUS_ID จะถูกนำมาตรวจสอบหาชื่อไดรฟ์เวอร์โดยเทียบจากไฟล์ driverid.list ซึ่งมีตัวอย่างดังนี้ (MBUS_ID : ชื่อไดรฟ์เวอร์)

AdvancedStereo400:com.home.service.driver.estereo

AdvancedStereo500:com.home.service.driver.estereo

โดยไฟล์ driverid.list นี้จะเก็บอยู่บนเซิร์ฟเวอร์ของ Home-Connected inc. และเป็นไปได้ที่ MBUS_ID หลายค่าจะใช้ไดรฟ์เวอร์ตัวเดียวกัน หากตรวจสอบพบ MBUS_ID ก็จะคืนค่าชื่อไดรฟ์เวอร์ซึ่งอยู่ในรูปแบบแพ็คเกจของจาวา ชื่อไดรฟ์เวอร์ที่ได้จะถูกนำมาตรวจสอบหา URL ของไดรฟ์เวอร์โดยเทียบจากไฟล์ driverurls.list ซึ่งมีตัวอย่างดังนี้ (ชื่อไดรฟ์เวอร์:URL)

com.home.service.driver.estereo:http://192.168.5.114/drivers/estereo.jar

ถ้าหากตรวจสอบพบชื่อไดรฟ์เวอร์ ฟังก์ชัน findDrivers ก็จะคืนค่า URL ของไดรฟ์เวอร์กลับมา จากนั้นค่า URL นี้จะถูกส่งไปให้ฟังก์ชัน loadDrivers ทำการดาวน์โหลดไดรฟ์เวอร์มาติดตั้งบนเกตเวย์

ในกรณีที่ไดรฟ์เวอร์ ของอุปกรณ์ที่ทำงานบนโปรโตคอล MBus ไม่ได้ถูกพัฒนาขึ้นเองโดย Home-Connected inc. ผู้ผลิตอุปกรณ์ต้องมอบข้อมูล MBUS_ID ชื่อไดรฟ์เวอร์ รวมทั้ง URL ของไดรฟ์เวอร์ให้ Home-Connected inc. ทำการเพิ่มลงในไฟล์ driverid.list และdriverurls.list ข้างต้น

4.1.1.3 การติดต่อกับอุปกรณ์

เมื่อได้รับการเชื่อมต่อจากอุปกรณ์ MBus จะแตก thread ไปดูแลและสร้างอ็อบเจกต์ของคลาส MBusServiceImpl มาใช้ในแยกแยะและทำการติดต่อระหว่างเกตเวย์กับอุปกรณ์ดังที่กล่าวมาแล้วข้างต้น คลาส MBusServiceImpl เป็นคลาสที่สืบทอดคุณสมบัติมาจาก Device interface ของเฟรมเวิร์ก OSGI ซึ่งใช้ในแยกแยะ device service (เซอร์วิสที่ลงทะเบียนด้วยแพ็คเกจ org.osgi.service.device.Device)

ในการแยกแยะอุปกรณ์นั้น อุปกรณ์ที่เข้ามาเชื่อมต่อต้องส่งอ็อบเจกต์ของ MBus ID มาให้ MBusServiceImpl ในตอนเริ่มต้นการเชื่อมต่อ มิฉะนั้นเกตเวย์จะไม่สนใจการเข้ามาเชื่อมต่อของอุปกรณ์นั้นๆ เมื่อได้รับส่งอ็อบเจกต์ของ MBus ID แล้ว MBusServiceImpl จะนำข้อมูล ID ในอ็อบเจกต์ออกมาซึ่งในโครงงานนี้สเตอริโอจะมี ID คือ “AdvancedStereo400” ใช้ในการลงทะเบียน device service ของสเตอริโอ จากนั้นเซอร์วิส DriverLocator ทุกตัวในเฟรมเวิร์ก รวมทั้งเซอร์วิส DriverLocator ของ MBus เองก็จะถูกเรียกให้หาและดาวน์โหลดไดรฟ์เวอร์สำหรับสเตอริโอดังที่กล่าวมาแล้วข้างต้น

เมื่อติดต่อไดรฟ์เวอร์สำหรับสเตอริโอบนเกตเวย์แล้ว สเตอริโอนี้ก็จะพร้อมใช้งาน การติดต่อที่จะเกิดขึ้นต่อไประหว่าง MBus และสเตอริโอจะถูกจัดการโดย thread ของคลาส MBusServiceImpl

Mbus เป็นโปรโตคอลแบบ “event based” กล่าวคือหากอุปกรณ์หรือเซอร์วิสต่างๆในเกตเวย์ต้องการส่งและรับ message จะต้องทำการเพิ่มอ็อบเจกต์ MBusPacketListener รอฟัง message ที่ส่งมาผ่าน MBus ซึ่งสามารถกระทำผ่านฟังก์ชัน addListener ของคลาส MBusServiceImpl

เมื่อไหร่ก็ตามที่มี message เข้ามา ไดรฟ์เวอร์ของ Mbus จะเรียกฟังก์ชัน HandleMessage ของทุกๆ MBusPacketListener พร้อมส่งอ็อบเจกต์ของ MBusMessage ไปให้ อุปกรณ์และเซอร์วิสต่างๆ จะพิจารณา MBusMessage ที่ได้รับว่าส่งถึงตัวเองหรือไม่ ถ้าใช่ก็จะนำ message มาทำงานต่อไป

4.1.2 ส่วนติดต่อกับผู้ใช้งาน (User Interface)

เนื่องจากเกตเวย์ถูกออกแบบให้เป็นกล่องรับสัญญาณ ดังนั้นจึงจำเป็นต้องมีอุปกรณ์ติดต่อกับผู้ใช้งานเป็นรีโมทคอนโทรล ซึ่งทำการจำลองโดยใช้ไลบรารี Swing ในจาวา

4.1.2.1 การจำลองอุปกรณ์รีโมทคอนโทรล

ส่วนหน้าต่างของรีโมทคอนโทรลนั้น พัฒนาโดยใช้ ไลบรารี Swing ในจาวาสร้างอ็อบเจกต์ของคลาส JTabbedPane ซึ่งเป็นหน้าต่างว่างๆ ดังรูปที่ 4.1 ไปให้เซอร์วิสที่ต้องการแสดงผลบนรีโมทใช้งาน โดยเซอร์วิสที่ต้องการแสดงผลบนรีโมทจะสร้างอ็อบเจกต์ของ JPanel ใส่ไปบน JTabbedPane เพื่อแสดงข้อมูลบนรีโมท



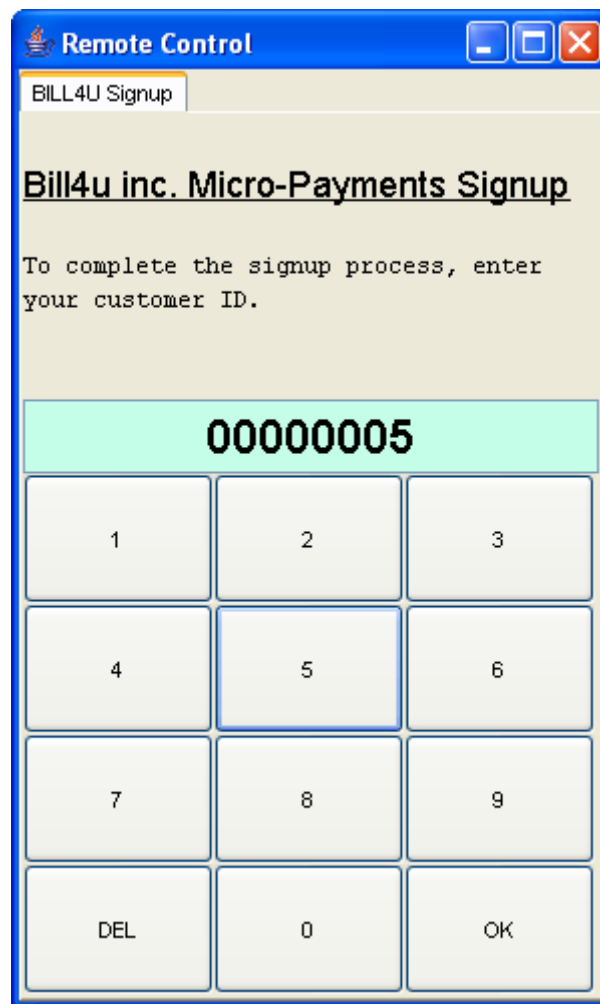
รูปที่ 4.1 หน้าต่างว่างๆ ของ JTabbedPane

เซอร์วิสที่รองรับการใช้งานรีโมทซึ่ง Home-Connected inc. คิดตั้งมาให้บนเกตเวย์นั้นจะเตรียมฟังก์ชันไว้ให้เซอร์วิสอื่นเรียกใช้งาน คือฟังก์ชันที่ใช้ลงทะเบียนอ็อบเจกต์ของ JPanel ใหม่บน JTabbedPane และฟังก์ชันที่ใช้ยกเลิกการลงทะเบียน JPanel เมื่อไม่ต้องการใช้งาน

4.1.2.2 เซอร์วิสที่รองรับการใช้งานรีโมทบนเกตเวย์

เซอร์วิสที่รองรับการใช้งานรีโมทซึ่ง Home-Connected inc. คิดตั้งมาให้บนเกตเวย์นั้นมีเรียกว่า “UIService” ทำหน้าที่ควบคุมการติดต่อกับผู้ใช้โดยทำการเพิ่มและลบ JPanel บน JTabbedPane ตามการใช้งานของเซอร์วิสอื่น ๆ ที่ต้องการติดต่อกับผู้ใช้ผ่านรีโมทคอนโทรล

เมื่อเซอร์วิสใด ๆ ต้องการแสดงข้อมูลใหม่บนรีโมท ก็จะต้องลงทะเบียน JPanel ใหม่บน JTabbedPane โดยใช้ฟังก์ชัน registerPanel ระบุชื่อแท็บที่แตกต่างกัน หากชื่อแท็บนั้นซ้ำ UIService จะปฏิเสธการลงทะเบียน JPanel นั้นๆ และแสดงข้อผิดพลาด UIServiceException เพื่อหลีกเลี่ยงเหตุการณ์นี้ แต่ละเซอร์วิสควรขึ้นต้นชื่อแท็บด้วยชื่อบริษัทที่ผลิตเซอร์วิสนั้นๆ ดังรูปที่



รูปที่ 4.2 การขึ้นต้นชื่อแท็บด้วยชื่อบริษัทที่ผลิตเซอร์วิส

เซอร์วิสที่ลงทะเบียนเป็น JPanel ใหม่บน JTabbedPane สามารถเพิ่มองค์ประกอบของ Java Swing เช่นปุ่มและข้อความที่ต้องการแสดงผลลงไปได้ เมื่อเซอร์วิสใช้การ JPanel เสร็จแล้วก็สามารถลบ JPanel เดิมบน JTabbedPane ได้โดยใช้ฟังก์ชัน unregisterPanel

4.1.3 ส่วนการจัดการเกตเวย์ (Gateway Management)

เซอร์วิสของการจัดการเกตเวย์ซึ่งทำงานอยู่บนเกตเวย์นั้น ทำหน้าที่รับผิดชอบการรับการร้องขอพิจารณาติดตั้งเซอร์วิสจากเซิร์ฟเวอร์ของผู้ให้บริการเกตเวย์ ก่อนที่เซอร์วิสที่ร้องขอติดตั้งจะถูกติดตั้งบนเกตเวย์ได้นั้น เซอร์วิสของการจัดการเกตเวย์จะต้องตรวจสอบว่าเซอร์วิสนั้นถูกแก้ไขไปจากเซอร์วิสที่ผ่านการรับรองหรือไม่ก่อน

ในขั้นตอนการตรวจสอบนี้ไม่รวมถึงการดาวน์โหลดไคลฟ์เวอร์ของอุปกรณ์ ซึ่งจะถูกดาวน์โหลดมาติดตั้งอัตโนมัติโดยเฟรมเวิร์กของ OSGI ไม่ได้ผ่านการติดตั้งโดยเซอร์วิสของการจัดการเกตเวย์

ดังที่กล่าวมาแล้วในบทที่ผ่านมา ในโครงการนี้การทำลายเซ็นดิจิทัลจะใช้แอปพลิเคชันที่สร้างไฟล์ “certificate” ของทั้งบันเดิลแทนการใช้เครื่องมือ jarsigner ซึ่งมีในจาวาสตริงไฟล์ “certificate” สำหรับแต่ละไฟล์ในบันเดิล

4.1.3.1 แอปพลิเคชัน DSAKeyGen

DSA KeyGen เป็นแอปพลิเคชันจาวาง่ายๆที่โครงการนี้พัฒนาขึ้นเพื่อสร้าง private key และ public key ใช้การเข้ารหัสแบบ DSA ในการนำไปใช้นั้น private key จะถูกใช้โดย Home-Connected inc. ในการทำลายเซ็นดิจิทัลเซอร์วิสเพื่อรับรองความถูกต้อง โดย private key จะต้องถูกเก็บเป็นความลับ ส่วน public key จะใช้โดยเซอร์วิสที่ทำหน้าที่จัดการในเกตเวย์ในการพิสูจน์เซอร์วิสที่ร้องขอติดตั้งว่าถูกแก้ไขภายหลังจากที่ได้รับการรับรองหรือไม่ public key จะถูกเก็บอยู่ในอุปกรณ์เกตเวย์แต่ละตัวก่อนส่งมอบให้ลูกค้า

4.1.3.2 แอปพลิเคชัน Signer

Signer เป็นแอปพลิเคชันที่ทำหน้าที่ทำลายเซ็นดิจิทัลสำหรับบันเดิลโดยใช้ DSA private key ที่ได้จากข้อ 4.1.3.1 จะได้ certificate ซึ่งจะถูกระบุที่หน้าที่จัดการในเกตเวย์พิสูจน์ในภายหลังโดยใช้ public key

Signer เป็นแอปพลิเคชันจาวาที่ต้องการ 2 พารามิเตอร์คือไฟล์ที่ต้องการทำลายเซ็นดิจิทัล และชื่อไฟล์ที่ต้องการเก็บ certificate

ในการเก็บ public key ในเกตเวย์นั้นและการเก็บ private key ในเซิร์ฟเวอร์ของ Home-Connected inc. นั้น เพื่อความปลอดภัยจึงทำการเก็บในรูปแบบ HomePrivateKey.class และ HomePublicKey.class แทนการเก็บเป็นไฟล์ตัวอักษรธรรมดา อีกทั้งยังสามารถเขียนโปรแกรมเรียกใช้ HomePrivateKey.class ในการทำลายเซ็นดิจิทัล และเรียกใช้ HomePublicKey.class ในการพิสูจน์เซ็นดิจิทัลได้

ในการทำลายเซ็นดิจิทัล แอปพลิเคชัน Signer จะสร้าง certificate ออกมาเป็นไฟล์ซึ่งบรรจุข้อมูลในรูปแบบไบนารี และไม่แปลงเก็บในรูปแบบ ASCII เนื่องจากการเพิ่มความปลอดภัยไม่ให้อ่านได้ แสดงตัวอย่าง certificate ได้ดังรูปที่ 4.3

0,000|0+-*09<0éÅñ:·5⁻1#0000_0๗๖>Y0600P060?8๐๔

รูปที่ 4.3 ตัวอย่าง certificate

เมื่อได้รับ certificate จากผู้ให้บริการเกตเวย์แล้ว ผู้ให้บริการเซอร์วิสจะไม่สามารถแก้ไขเซอร์วิสที่ถูกรับรองแล้วได้ หากเซอร์วิสที่ทำหน้าที่จัดการในเกตเวย์ตรวจพบว่าเซอร์วิสถูกแก้ไขก็จะไม่อนุญาตให้ติดตั้งลงเกตเวย์

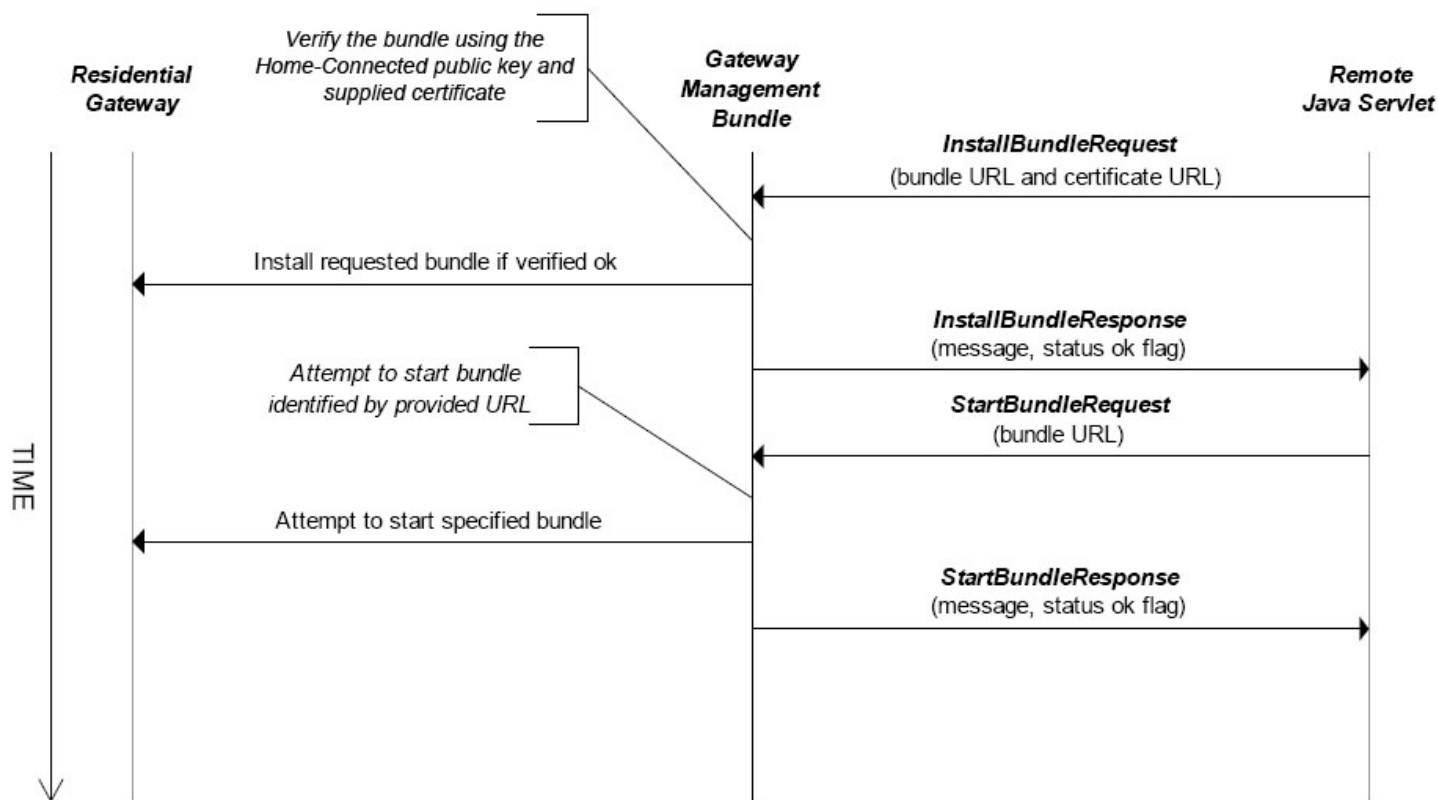
4.1.3.3 เซอร์วิสที่ทำหน้าที่จัดการในเกตเวย์

เซอร์วิสที่ทำหน้าที่จัดการในเกตเวย์ในโครงงานนี้มีชื่อว่าเซอร์วิส “manager” ซึ่งเป็นเซอร์วิสที่ทำงานแบบแบ่ง thread อยู่บนเกตเวย์โดย Home-Connected inc. จะติดตั้งเซอร์วิสนี้ลงบนเกตเวย์ทุกตัวก่อนส่งมอบให้ลูกค้า ทำหน้าที่รับการร้องขอพิจารณาติดตั้งเซอร์วิสมาจาก Servlet ชื่อ “InstallBundle” ของเซิร์ฟเวอร์ Home-Connected inc.

เซอร์วิส “manager” จะรอฟังการติดต่อจากเซิร์ฟเวอร์อยู่ที่พอร์ต 888 เมื่อมีการเชื่อมต่อเข้ามา เซอร์วิส “manager” จะแตก thread ไปจัดการกับการเชื่อมต่อนั้น โดย Servlet ชื่อ “InstallBundle” จะส่งอ็อบเจกต์ของคลาส InstallBundleRequest มายังเซอร์วิส “manager” บนเกตเวย์เพื่อร้องขอพิจารณาติดตั้งเซอร์วิส ในอ็อบเจกต์ของคลาส InstallBundleRequest จะบรรจุ URL ของบันเดิลเซอร์วิสที่ต้องการติดตั้ง และ URL ของไฟล์ certificate ที่ออกโดย Home-Connected inc.

เซอร์วิส “manager” จะทำการดาวน์โหลดบันเดิลและไฟล์ certificate จาก URL ข้างต้นมาทำการพิสูจน์ลายเซ็นดิจิทัล ว่าเซอร์วิสถูกแก้ไขไปจากเซอร์วิสที่ถูกทำการรับรองไปหรือไม่ โดยใช้ public key ของ Home-Connected inc. ที่เก็บอยู่ในไฟล์ HomePublicKey.class ในเกตเวย์ หากพิสูจน์ลายเซ็นดิจิทัลแล้วพบว่าถูกต้อง ก็จะทำการติดตั้งบันเดิลนี้ลงบนเกตเวย์ และส่งอ็อบเจกต์ของคลาส InstallBundleResponse กลับไปยังเซิร์ฟเวอร์ระบุผลลัพธ์การติดตั้ง

แม้เซอร์วิสจะถูกติดตั้งบนเกตเวย์แล้ว แต่จะยังไม่เริ่มทำงานจนกว่าเซอร์วิส “manager” จะได้รับส่งอ็อบเจกต์ของคลาส StartBundleRequest จากเซิร์ฟเวอร์ซึ่งส่งมาหลังการติดตั้งเสร็จสมบูรณ์ อ็อบเจกต์ของคลาส StartBundleRequest จะบรรจุ URL ของบันเดิลในเกตเวย์ที่ต้องการสั่งให้เริ่มทำงาน เซอร์วิส “manager” จะใช้ URL ของบันเดิลนี้ทำงานร่วมกับเฟรมเวิร์ก OSGI ในการสั่งงานบันเดิล จากนั้นจะส่งอ็อบเจกต์ของคลาส StartBundleResponse กลับไปยังเซิร์ฟเวอร์เพื่อบอกผลลัพธ์การเริ่มทำงานของบันเดิล แสดงการส่งข้อมูลได้ดังรูปที่ 4.4



รูปที่ 4.4 การส่งข้อมูลในการร้องขอติดตั้งและเริ่มการทำงานของเซอร์วิส

4.1.3.4 ส่วนที่ใช้รองรับการร้องขอติดตั้งเซอร์วิส (Web Installer Interface)

พัฒนาโดย Home-Connected inc. ในการรองรับการร้องขอติดตั้งเซอร์วิสจากผู้ให้บริการเซอร์วิสต่างๆ มาตรวจสอบข้อมูลลูกค้าที่สมัครใช้เซอร์วิสว่าถูกต้องหรือไม่ก่อนจะส่งไปให้เซอร์วิสที่ทำหน้าที่จัดการในเกตเวย์พิจารณาติดตั้งต่อไปดังที่กล่าวมาแล้วในหัวข้อ 4.1.3.3

ในการใช้งานระบบ ผู้ให้บริการเซอร์วิสต้องมีเว็บเพจHTML ที่ทำหน้าที่ส่งการร้องขอติดตั้งเซอร์วิสมายังส่วนที่ใช้รองรับการร้องขอติดตั้งเซอร์วิสของ Home-Connected inc. ซึ่งในการพัฒนานั้นประกอบด้วย Servlet ชื่อ “ConfirmInstall” ทำหน้าที่ตรวจสอบว่าลูกค้าที่สมัครใช้เซอร์วิสเป็นเจ้าของเกตเวย์ที่ระบุหรือไม่ หากตรวจสอบพบว่าถูกต้องก็จะแสดงหน้าเว็บให้ลูกค้ากรอกรหัสผ่านแล้วส่งไปให้ Servlet ชื่อ “InstallBundle” ทำหน้าที่ตรวจสอบว่าลูกค้ากรอกรหัสผ่านถูกต้องหรือไม่ หากถูกต้องก็จะส่งการร้องขอพิจารณาติดตั้งเซอร์วิสไปยังเซอร์วิสที่ทำหน้าที่จัดการภายในเกตเวย์ต่อไป

ในส่วนของข้อมูลที่ต้องการจากจากลูกค้าที่สมัครใช้เซอร์วิส ขึ้นกับแต่ละผู้ให้บริการเซอร์วิส แต่ Home-Connected inc. ระบุว่าต้องมีข้อมูลที่เป็นดังตารางที่ 4.1

ชื่อ field	รายละเอียด
providername	ชื่อผู้ผลิตเซอร์วิส
servicename	ชื่อของเซอร์วิส
Bundleurl	URL ของเซอร์วิส
Certurl	URL ของไฟล์ certificate ของเซอร์วิส
Successurl	URL ของเว็บเพจที่จะแสดงกรณีติดตั้งสำเร็จ
Errorurl	URL ของเว็บเพจที่จะแสดงกรณีติดตั้งไม่สำเร็จ
Custno	หมายเลขลูกค้าที่ได้จากผู้ให้บริการเกตเวย์
Serialno	หมายเลขประจำเกตเวย์ที่ต้องการติดตั้งเซอร์วิส

ตารางที่ 4.1 ข้อมูลที่จำเป็นต้องส่งให้ส่วนที่ใช้รองรับการร้องขอติดตั้งเซอร์วิส
(Web Installer Interface)

ผู้ให้บริการเซอร์วิสต้องแน่ใจว่าลูกค้ากรอกข้อมูล หมายเลขลูกค้าและหมายเลขประจำเกตเวย์ตอนสมัครผ่านเว็บ ส่วนข้อมูลอื่น ๆ นั้นระบบสามารถคำนวณได้อัตโนมัติโดยไม่ต้องให้ลูกค้ากรอก

สามารถแสดงตัวอย่างรูปแบบหน้าเว็บการสมัครใช้เซอร์วิสของผู้ให้บริการเซอร์วิสดังรูปที่ 4.5 และตัวอย่างแสดงหน้าเว็บให้ลูกค้ากรอกรหัสผ่านยืนยันการติดตั้งเซอร์วิสดังรูปที่ 4.6

MUSIC CONNECTED
Powered by Sothink

SIGN UP

To immediately sign-up for our cool Streaming Media service (*MediaStream*) fill out the following form and press the 'Submit' button.

You will then be taken to your gateway operators web site where, upon confirmation of your sign-up, the software will be automatically installed onto your residential gateway.

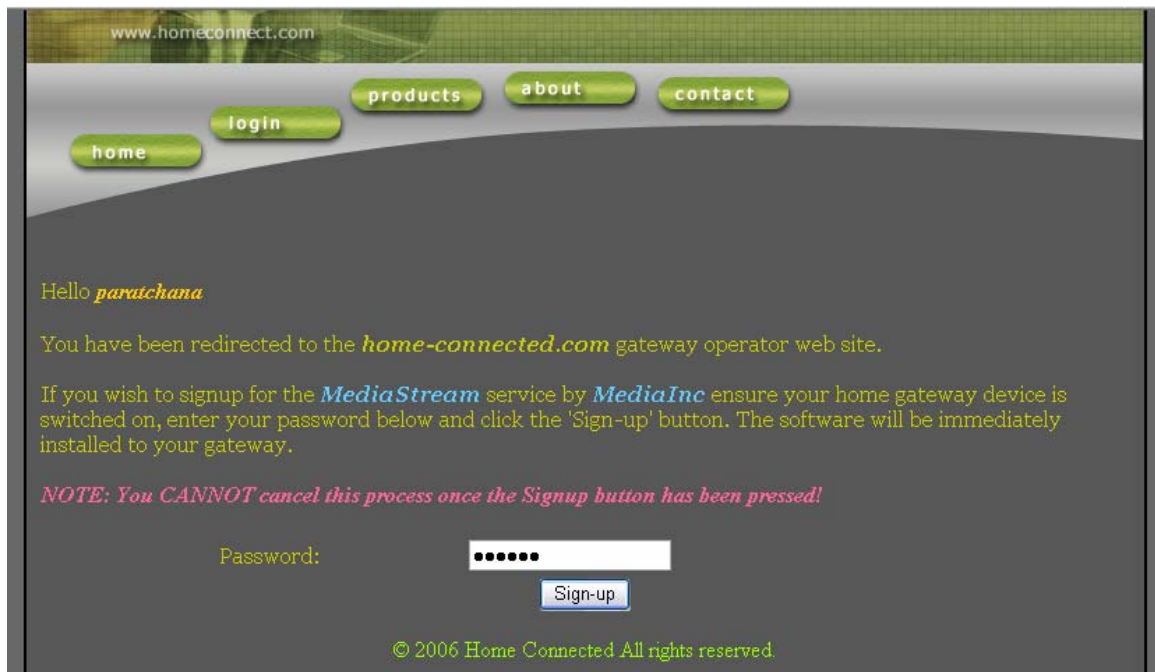
Note that Customer Number is the one you received from your gateway operator.

Customer Number:

Residential Gateway Serial Number:

You should be able to find your residential gateway serial number on the back of the unit.

รูปที่ 4.5 ตัวอย่างรูปแบบหน้าเว็บการสมัครใช้เซอร์วิสของผู้ให้บริการเซอร์วิส



รูปที่ 4.6 ตัวอย่างหน้าเว็บให้ลูกค้ากรอกรหัสผ่านยืนยันการติดตั้งเซอร์วิส

จากรูปที่ 4.5 และ 4.6 หากตรวจสอบพบว่าลูกค้าที่สมัครใช้เซอร์วิสเป็นเจ้าของเกตเวย์ที่ระบุและกรอกรหัสผ่านถูกต้อง InstallBundle Servlet จะดึงข้อมูลหมายเลขไอพีของเกตเวย์ที่ต้องการติดตั้งเซอร์วิสจากตาราง gatewaybox ในฐานข้อมูลมาใช้ในการส่งการร้องขอพิจารณาติดตั้งไปยังเซอร์วิสที่ทำหน้าที่จัดการภายในเกตเวย์ต่อไป

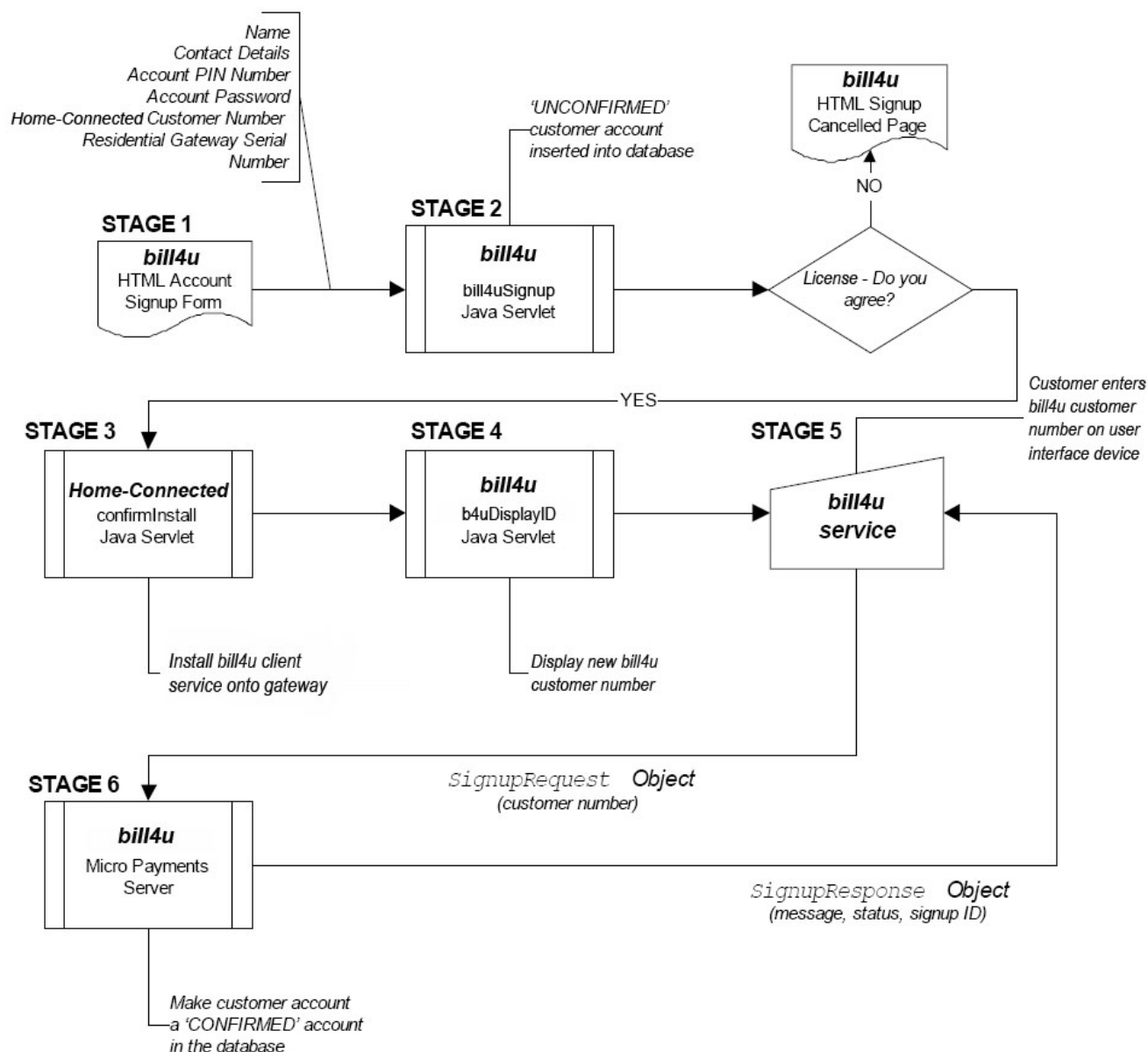
และเมื่อได้รับผลตอบกลับจากเกตเวย์ InstallBundle Servlet ก็จะโอนลูกค้าไปยังหน้าเว็บแสดงผลลัพธ์การติดตั้งตาม Successurl และ Errorurl ที่ผู้ให้บริการเซอร์วิสส่งมาให้

4.1.4 ระบบให้บริการของ Bill4u inc.

ในโครงการนี้ได้ออกแบบให้ลูกค้าชำระค่าบริการใช้เซอร์วิสตามอัตราค่าใช้บริการตามการใช้งานจริง โดยมี Bill4u inc. เป็นองค์กรกลางที่รับผิดชอบการเก็บค่าบริการ ระบบให้บริการของ Bill4u inc. ประกอบด้วย 2 ส่วนคือแอปพลิเคชันฝั่งเซิร์ฟเวอร์และเซอร์วิสที่ทำงานบนเกตเวย์

4.1.4.1 ขั้นตอนการสมัครใช้เซอร์วิส Micropay

ขั้นตอนการสมัครใช้เซอร์วิส Micropay มี 6 ขั้นตอนดังรูปที่ 4.7 ซึ่งในการพัฒนาจะต้องสร้างเว็บเพจ HTML และ Java Servlets ที่รองรับการทำงานในขั้นตอนต่างๆ



รูปที่ 4.7 ขั้นตอนการสมัครใช้เซอร์วิส Micropay

แต่ละขั้นตอนมีการทำงานดังนี้

1. ลูกค้าสมัครใช้เซอร์วิสผ่านเว็บไซต์ของ Bill4u inc. โดยกรอกข้อมูลชื่อ ที่อยู่หมายเลขลูกค้าและรหัสผ่านที่ได้รับจาก Home-Connected inc. หมายเลขประจำเครือข่าย และรหัสพินในการยืนยันการชำระค่าบริการ ซึ่งข้อมูลเหล่านี้จะถูกส่งไปให้ bill4uSignup Servlet
2. bill4uSignup Servlet บันทึกข้อมูลลูกค้าใหม่ลงในตาราง customer ในฐานข้อมูล โดยให้ฟิลด์ "signupid" มีค่าเริ่มต้นเป็น 0 แสดงสถานะ "UNCONFIRMED" ของลูกค้า พร้อมทั้งแสดงเงื่อนไขการใช้งานเซอร์วิสให้ลูกค้าพิจารณายอมรับเงื่อนไข จากนั้นจะส่งการร้องขอติดตั้ง

เซอวิสไปให้ส่วนรองรับการร้องติดตั้งเซอวิสของ Home-Connected inc. ทำการตรวจสอบติดตั้งเซอวิสต่อไป

3. ส่วนรองรับการร้องติดตั้งเซอวิสของ Home-Connected inc. พิจารณาติดตั้งเซอวิส Micropay ไปยังเกตเวย์ภายในบ้าน หากติดตั้งสำเร็จลูกค้าจะถูกโอนกลับมายัง b4uDisplayID Servlet ของBill4u inc.

4. b4uDisplayID Servlet แจ้งผลการติดตั้งเซอวิสและแสดงหมายเลขลูกค้าของ Bill4u inc. ให้ลูกค้าใส่ในรีโมทคอนโทรลเพื่อทำการตั้งค่าในการใช้งานครั้งแรก

5. เซอวิส Micropay ในเกตเวย์จะทำงาน และขึ้นหน้าจอให้ลูกค้าใส่หมายเลขลูกค้าของ Bill4u inc. 8 หลักในรีโมท ซึ่งจะถูส่งมาให้ แอปพลิเคชัน MP Server ทางฝั่งเซิร์ฟเวอร์ของ Bill4u inc. ในรูปแบบอ็อบเจกต์ของ SignupRequest

6. แอปพลิเคชัน MP Server ทางฝั่งเซิร์ฟเวอร์ตรวจสอบ SignupRequest ที่ได้รับมาว่า หมายเลขลูกค้าถูกต้องหรือไม่ ก่อนจะสร้างชุดของตัวเลข signup ID เป็ปลงในฟิลด์ signupid ของตาราง customer และส่ง signup ID นี้กลับไปยังเซอวิส Micropay ในเกตเวย์ในรูปแบบของอ็อบเจกต์ SignupResponse เซอวิส Micropay จะสร้างไฟล์ที่เก็บการตั้งนี้ไว้ในเกตเวย์

4.1.4.1 เซอวิส Micropay บนเกตเวย์

คลาส MicroPaymentServiceImp จะถูกเรียกใช้จากเซอวิส Mediastream ในเกตเวย์ให้ทำหน้าที่สร้างการร้องขอชำระค่าบริการส่งไปให้แอปพลิเคชัน MP Server ทางฝั่งเซิร์ฟเวอร์ และส่งการตอบกลับไปยังเซอวิส Mediastream

เมื่อเริ่มทำงาน เซอวิส Micropay บนเกตเวย์จะตรวจสอบหาไฟล์ที่เก็บการตั้งค่าเซอวิส (configuration file) ซึ่งควรมีเก็บไว้ในเกตเวย์หลังการตั้งค่าเพื่อใช้งานครั้งแรก ไฟล์ที่เก็บการตั้งค่าเซอวิสในที่นี้ชื่อว่า customerData บรรจุหมายเลขลูกค้าและ signup ID รวมทั้งวันเวลาที่ตั้งค่าเพื่อใช้งานครั้งแรกด้วย

ในการส่งการร้องขอชำระค่าบริการนั้น เซอวิส Micropay จะส่ง signup ID ไปให้แอปพลิเคชัน MP Server ทางฝั่งเซิร์ฟเวอร์เพื่อเปรียบเทียบกับฟิลด์ signupid ในฐานข้อมูลว่าตรงกันหรือไม่ก่อนจะหักบัญชีลูกค้า กระบวนการนี้มีเพื่อป้องกันไม่ให้เจ้าของเกตเวย์บ้านอื่นแอบนำบัญชีที่ไม่ใช่ของตนเองไปชำระค่าบริการ แม้จะรู้รหัสพินก็ตาม

ไฟล์ที่เก็บการตั้งค่าเซอวิสแสดงได้ดังตัวอย่างข้างล่างนี้

```
#Micropayment Signup Data
```

```
#Mon Feb 20 19:49:04 ICT 2006
```

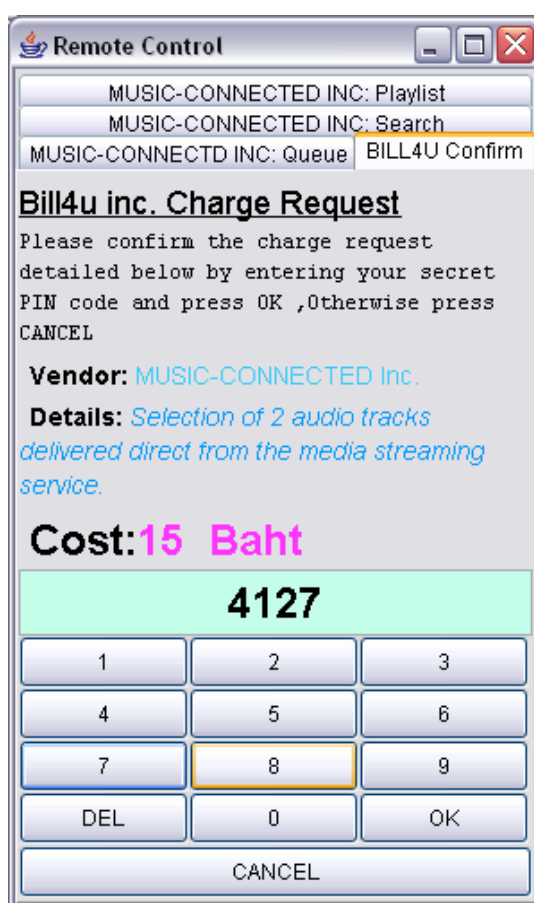
```
signupID=143294347835870297
```

```
custID=00000052
```

เซอร์วิส Micropay ต้องสามารถติดต่อกับอุปกรณ์รีโมทคอนโทรลเพื่อรับค่าหมายเลขพินจากผู้ใช้ และแสดงผลการชำระเงินได้

ในการที่เซอร์วิส Mediastream จะสามารถติดต่อเซอร์วิส Micropay เพื่อร้องขอชำระเงินได้นั้น จะต้องอ้างอิงถึงเซอร์วิส Micropay โดยใช้ฟังก์ชัน getServiceReference ของเฟรมเวิร์กซึ่งจะคืนค่าเป็นอ็อบเจกต์ที่อ้างอิงถึงเซอร์วิส Micropay กลับมาให้เรียกใช้เซอร์วิสได้โดยใช้ฟังก์ชัน GetService ซึ่งภายในเซอร์วิส Micropay จะมีฟังก์ชันเดียวให้เรียกใช้งานคือฟังก์ชัน makeCharge

เมื่อฟังก์ชัน makeCharge ถูกเรียก เซอร์วิส Micropay จะติดต่อกับผู้ใช้โดยสร้าง JPanel ใหม่ขึ้นบนรีโมทแสดงรายละเอียดการชำระค่าบริการ (ซึ่งได้รับมาจากเซอร์วิส Mediastream) และแจ้งให้ลูกค้าใส่รหัสพิน 4 หลักเพื่อยืนยันการชำระเงินดังแสดงได้ในรูปที่ 4.3



รูปที่ 4.8 หน้าต่างแจ้งให้ลูกค้าใส่รหัสพินเพื่อยืนยันการชำระเงิน

เมื่อลูกค้าใส่รหัสพินแล้วกดปุ่ม OK เซอร์วิส Micropay จะสร้างอ็อบเจกต์ของคลาส ChargeRequest แล้วส่งไปให้แอปพลิเคชัน MPSTServer ที่เซิร์ฟเวอร์ตรวจสอบ เซิร์ฟเวอร์จะส่งผลการชำระเงินกลับมาเป็นอ็อบเจกต์ของคลาส ChargeResponse จากนั้นเซอร์วิส Micropay จะส่งผลลัพธ์กลับไปบอกเซอร์วิส Mediastream

4.1.4.2 แอปพลิเคชัน MPServer ทางฝั่งเซิร์ฟเวอร์

แอปพลิเคชัน MPServer ที่เซิร์ฟเวอร์เป็นจาวาแอปพลิเคชันที่มีการแตก thread ไปจัดการการเชื่อมต่อที่มาจากเครือข่าย รับผิดชอบการตรวจสอบการชำระเงิน และส่งผลลัพธ์กลับเครือข่าย

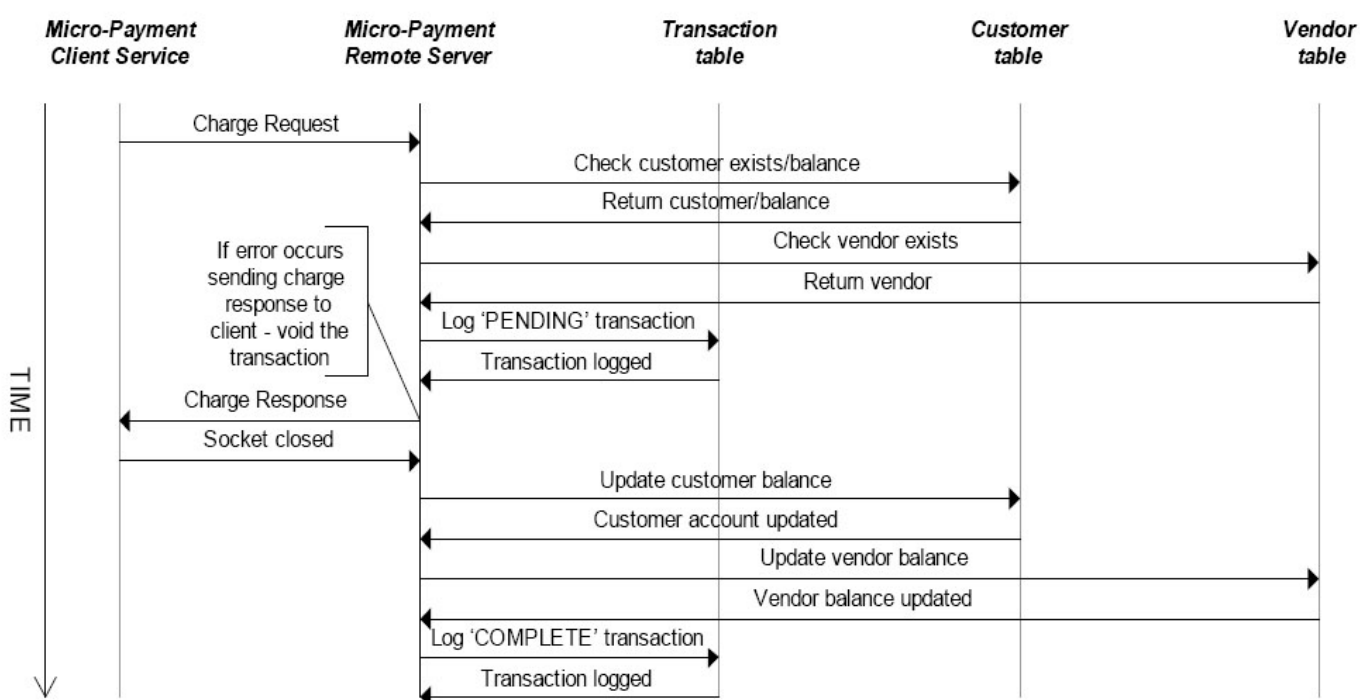
4.1.4.2.1 การจัดการการร้องขอชำระเงิน (Charge Request)

ทุกๆการร้องขอชำระเงินที่ส่งมาจากเครือข่ายต้องประกอบด้วยพารามิเตอร์ดังตารางที่ 4.2

ชื่อ field	รายละเอียด
ChargeInfo	รายละเอียดการชำระเงิน
venderID	ID ของบริษัทผู้ผลิตเซอร์วิส
venderName	ชื่อบริษัทผู้ผลิตเซอร์วิส
chargeAmount	จำนวนเงินที่ชำระ
acctNumber	หมายเลขลูกค้าได้จากไฟล์ที่เก็บการตั้งค่า
pinNumber	รหัสพินที่ลูกค้ากรอก
signupID	Signup ID ที่ได้จากไฟล์ที่เก็บการตั้งค่า

ตารางที่ 4.2 พารามิเตอร์ที่จำเป็นในการร้องขอชำระบริการ

สามารถแสดงการส่งข้อมูลของระบบการให้บริการของ Bill4u inc. ได้ดังรูปที่ 4.9



รูปที่ 4.9 การส่งข้อมูลของระบบการให้บริการของ Bill4u inc.

เซิร์ฟเวอร์จะพยายามตรวจสอบและบันทึกข้อมูลลงในตาราง transaction โดยเริ่มแรกจะตรวจสอบข้อมูลลูกค้ารวมทั้งยอดเงินคงเหลือ และตรวจสอบข้อมูลผู้ให้บริการเซอร์วิส จากนั้นจะเพิ่มข้อมูลลงในตาราง transaction โคนให้มีสถานะเป็น “PENDING” จากนั้น chargeResponse จะถูกส่งกลับไปยังเซอร์วิส Micropay บนเกตเวย์ เซิร์ฟเวอร์จะอัปเดตยอดเงินของลูกค้าและผู้ให้บริการเซอร์วิสและเปลี่ยนสถานะ transaction เป็น “COMPLETE”

4.1.5 ระบบให้บริการของ Music-Connected inc.

เนื่องจากการดาวน์โหลดเพลงมายังสเตอริโอในบ้าน ต้องอาศัยหลายองค์ประกอบทำงานร่วมกัน ในโครงการนี้จึงทำการพัฒนาส่วนนี้เป็นส่วนสุดท้าย ระบบให้บริการของ Music-Connected inc. มีองค์ประกอบดังนี้

4.1.5.1 การเพิ่มเพลงลงในฐานข้อมูล

ใช้โปรแกรม AddMedia ซึ่งพัฒนาโดยใช้จาวา และใช้แพ็คเกจ VorbisInfo ช่วยในการถอดข้อมูลจากส่วน header ของไฟล์รูปแบบ OGG โดย AddMedia จะทำหน้าที่เก็บข้อมูลเพลงลงในฐานข้อมูลโดยอัตโนมัติ โดยโครงสร้างฐานข้อมูลเพลงได้กล่าวไว้แล้วในบทที่ 3

โปรแกรม AddMedia นี้จะแกะข้อมูลเพลงจากส่วนของ header และทำการคำนวณสร้างชุดของตัวเลขสำหรับชื่อเพลง ชื่อนักร้อง ชื่ออัลบั้ม และประเภทเพลงให้โดยอัตโนมัติ เก็บลงในฐานข้อมูลพร้อมราคาเพลงที่ได้เป็นพารามิเตอร์ตอนเพิ่มเพลง

4.1.5.2 แอปพลิเคชัน MediaServer ทางฝั่งเซิร์ฟเวอร์

แอปพลิเคชัน MediaServer เป็นจาวาแอปพลิเคชันที่มีการแตก thread ไปจัดการการเชื่อมต่อที่มาจากเกตเวย์ ทำหน้าที่ค้นหาเพลงจากฐานข้อมูลโดยใช้คำสั่ง ‘SELECT LIKE xxx’ ในการค้นหาเพลงที่มีชุดของตัวเลขในฟิลด์ที่กำหนดตรงกับที่ลูกค้าส่งการร้องขอมา ผลลัพธ์การค้นหาที่เซิร์ฟเวอร์ส่งกลับไปยังเกตเวย์จะประกอบด้วยฟิลด์ trackid, artist, title, cost, genre, album และ URL ของไฟล์เพลงที่จะใช้ดาวน์โหลด ซึ่งจะถูกสร้างให้อยู่ในรูปแบบอ็อบเจกต์ของคลาส SearchResponse เพื่อส่งกลับไปยังเซอร์วิส Mediastream ในเกตเวย์

4.1.5.3 เซอร์วิส Mediastream ในเกตเวย์

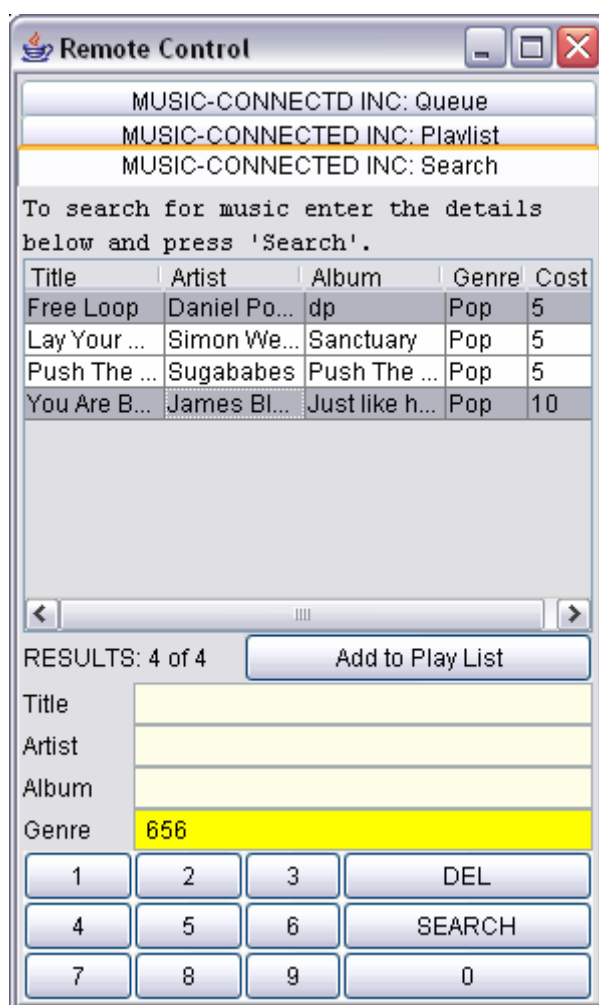
เซอร์วิส Mediastream ในเกตเวย์นี้เป็นองค์ประกอบหลักของโครงการ ซึ่งต้องทำงานติดต่อกับอุปกรณ์รีโมทคอนโทรล ไรร์ฟเวอร์ของสเตอริโอ และเซอร์วิส Micropay บนเกตเวย์ ในการทำงานของเซอร์วิส Mediastream นั้นต้องสามารถรองรับการค้นหาจากลูกค้า และส่งการร้องขอไปให้แอปพลิเคชัน MediaServer ในรูปแบบอ็อบเจกต์ของคลาส SearchRequest

เมื่อได้รับอ็อบเจกต์ของคลาส SearchResponse ที่เซิร์ฟเวอร์ตอบกลับมา เซอร์วิส Mediastream จะต้องนำผลลัพธ์ที่ได้มาแสดงให้ลูกค้าเลือกเพลงที่ต้องการฟังจากหน้าต่างบนรีโมท เมื่อลูกค้าเลือกเพลงได้แล้วเซอร์วิส Mediastream จะส่งการร้องขาระดับบริการไปให้

เซอร์วิส Micropay บนเกตเวย์ และรอฟังผลลัพธ์ก่อนจะดาวน์โหลดเพลงมาให้ไคลเอนต์ของสเตอร์ไอต่อไป

ส่วนติดต่อกับผู้ใช้งานของเซอร์วิส Mediastream นี้ประกอบด้วย 3 panels คือ MUSIC-CONNECTED INC:Search, MUSIC-CONNECTED INC:Playlist และ MUSIC-CONNECTED INC:Queue ซึ่งใช้ในการค้นหาเพลง เพิ่มเพลงในรายการเพลงเพื่อชำระค่าบริการ และแสดงคิวเพลงตามลำดับ

แสดงหน้าต่างการค้นหาเพลงได้ดังรูปที่ 4.10



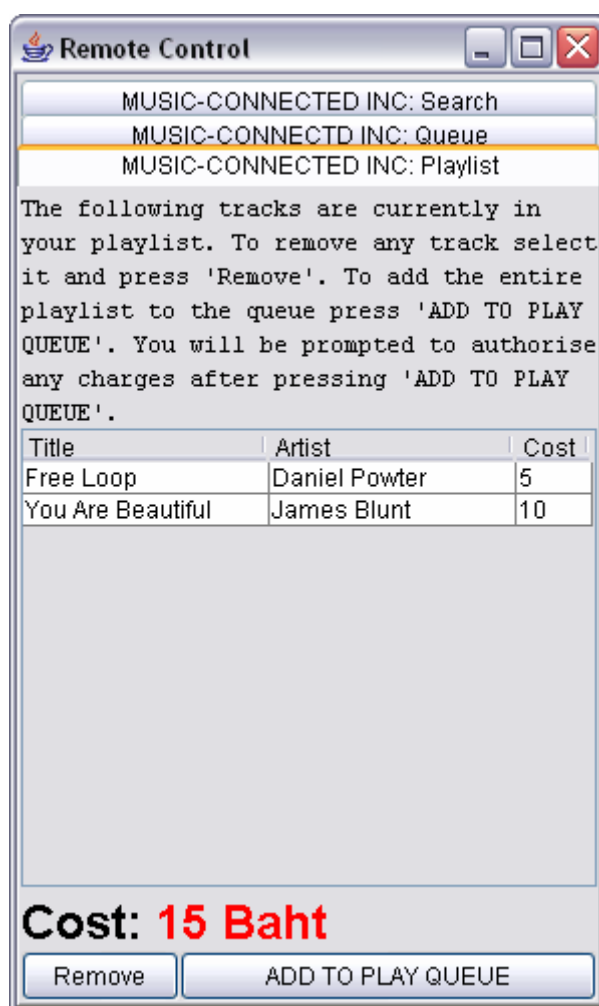
รูปที่ 4.10 หน้าต่างการค้นหาเพลง

ผู้ใช้งานสามารถค้นหาเพลงโดยกดเลือกฟิลด์ที่ต้องการค้นหา กดตัวเลขซึ่งใช้แทนข้อความที่ใช้ค้นหา แล้วกดปุ่ม “SERACH” โดยจะแสดงผลลัพธ์ของเพลงที่ค้นหานี้โมท

ในส่วนของหน้าต่างการแสดงผลนั้นพัฒนาโดยใช้คลาส JTable ใน Java Swing ซึ่งแสดงผลในรูปแบบของตาราง และผู้ใช้สามารถกดเลือกมากกว่า 1 เพลงในการฟังครั้งเดียวกันได้ เพลงที่ถูกเลือกจะแสดงเป็นสีเทา ดังรูป 4.10

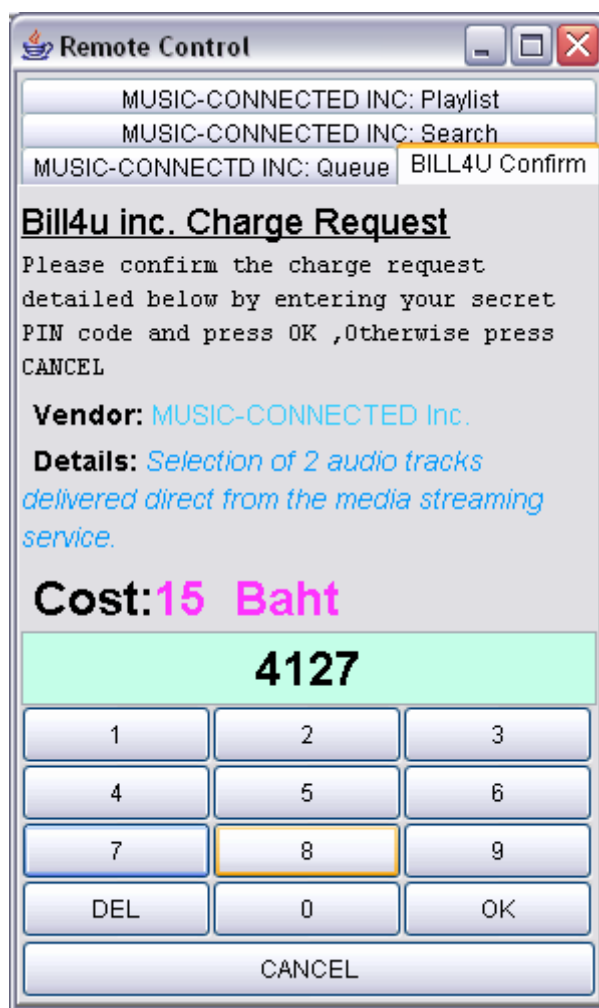
เมื่อผู้ใช้เลือกเพลงที่ต้องการ แล้วกดปุ่ม “Add to Play List” จะพบว่าเพลงที่ถูกเลือกจะถูกนำไปไว้ในหน้าต่าง MUSIC-CONNECTED INC:Playlist ซึ่งผู้ใช้จะถูกแจ้งให้ทราบถึงค่าบริการดาวน์โหลดเพลงที่ได้ทำการเลือกไว้

ซึ่งผู้ใช้อย่างสามารถลบเพลงที่ไม่ต้องการออกจากรายการเพลงได้ หรือกดปุ่ม “ADD TO PLAY QUEUE” ดังรูปที่ 4.11



รูปที่ 4.11 หน้าต่างรายการเพลง

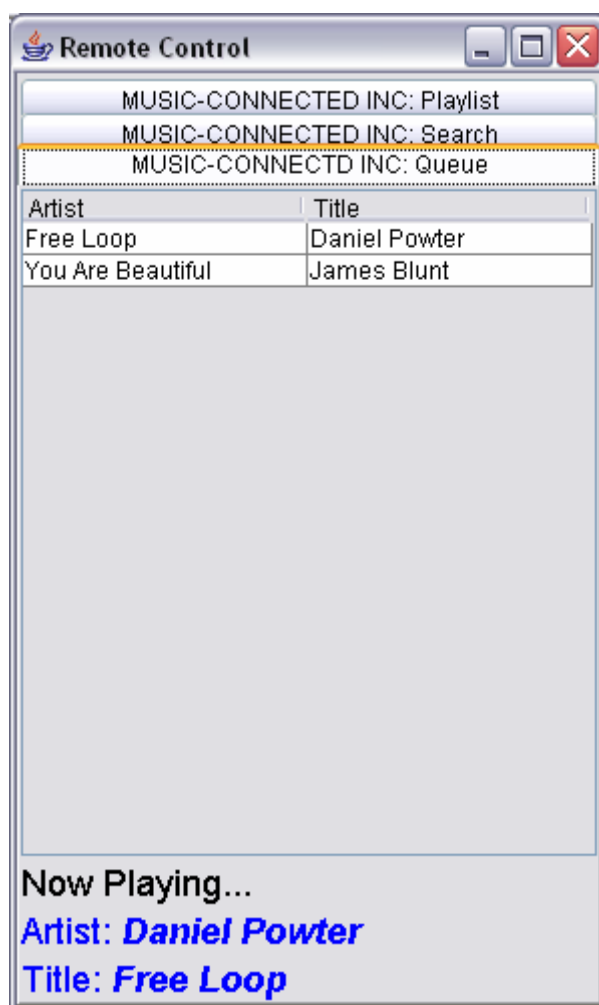
เมื่อกดปุ่ม “ADD TO PLAY QUEUE” แล้วจะเป็นการเรียกใช้เซอร์วิส Micropay บนเกตเวย์ให้ทำงาน และแสดงหน้าต่างชื่อ “BILL4U CONFIRM” เพื่อให้ผู้ใช้กรอกรหัสพินยืนยันการใช้เซอร์วิส ดังรูปที่ 4.12



รูปที่ 4.12 หน้าต่างผู้ใช้กรอกรหัสพินยืนยันการใช้เซอร์วิส

เมื่อเซอร์วิส Mediastream ได้รับการตอบกลับจากเซอร์วิส Micropay ว่าการชำระค่าบริการสำเร็จแล้ว ก็จะทำการดาวน์โหลดเพลงรูปแบบ OGG แล้วแปลงให้เป็นรูปแบบ PCM โดยใช้ไลบรารี VorbisSPI ในการพัฒนาจะส่งไปให้ไดรฟ์เวอร์ของสเตอริโอโดยการเรียกใช้ฟังก์ชัน PLAY ของไดรฟ์เวอร์สเตอริโอโดยส่งอ็อบเจกต์ AudioInputStreams ไปเป็นพารามิเตอร์

เมื่อเพลงถูกส่งไปเล่นยังสเตอริโอแล้ว ผู้ใช้สามารถตรวจสอบเพลงที่เล่นปัจจุบันและเพลงที่รออยู่ในคิวได้จากหน้าต่าง MUSIC-CONNECTED INC:Queue ซึ่งจะแสดงชื่อเพลงและชื่อนักร้องของเพลงที่กำลังเล่นดังแสดงในรูปที่ 4.13



รูปที่ 4.13 หน้าต่าง MUSIC-CONNECTED INC:Queue

4.1.6 ระบบสเตอริโอในบ้าน

ระบบสเตอริโอในบ้านมี 2 องค์ประกอบคือการจำลองอุปกรณ์รีโมทคอนโทรลภายนอก
 เกทเวย์ และไดรฟ์เวอร์สเตอริโอที่ติดตั้งภายในเกตเวย์

เนื่องจากโครงการนี้เน้นการพัฒนาเซิร์ฟวิซซึ่งทำงานร่วมกันบนเรสซิเดนเชียลเกตเวย์
 และไม่เน้นการพัฒนาฟังก์ชันการทำงานของอุปกรณ์สเตอริโอ ด้วยเวลาที่จำกัดในการพัฒนา
 ระบบ จึงออกแบบสเตอริโอให้มีหน้าที่แค่เพียงการเล่นเพลงที่ได้รับมาเท่านั้น

4.1.6.1 ไดรฟ์เวอร์สเตอริโอ

ไดรฟ์เวอร์สเตอริโอจะถูกไดรฟ์เวอร์ของ MBus คำนวณโหลดมาติดตั้งบนเกตเวย์ให้
 อัปเดตคั้งที่กล่าวมาแล้วในหัวข้อ 4.1.1.1 ไดรฟ์เวอร์สเตอริโอจะมีหน้าที่รับเพลงในรูปแบบ
 PCM จากเซิร์ฟวิซ Mediastream มาแปลงให้เป็นแพ็กเก็ต Mbus เพื่อส่งต่อไปยังสเตอริโอภายใน
 บ้าน

ไดร์ฟเวอร์สเตอริโอจะใช้โครงสร้าง LinkedList ในการจัดลำดับคิวเพลงในการเล่น ขณะที่อุปกรณ์สเตอริโอกำลังเล่นเพลงหนึ่งอยู่นั้น เมื่อมีรายการเพลงใหม่เข้ามา ก็จะถูกเก็บไว้ในคิวรอจนกระทั่งสเตอริโอเล่นเพลงก่อนหน้านี้เสร็จแล้ว

เซอร์วิสไดร์ฟเวอร์สเตอริโอประกอบด้วยฟังก์ชัน “play” ซึ่งมีพารามิเตอร์ 2 ตัวคือ AudioInputStream และ EStereoEventListener โดย AudioInputStream คือข้อมูลเพลง ส่วน EStereoEventListener จะเป็นอ็อบเจกต์ที่ถูกส่งมาให้ไดร์ฟเวอร์ใช้ตรวจสอบว่าทำการเล่นเพลงสำเร็จหรือไม่ เพื่อส่งผลลัพธ์กลับไปให้เซอร์วิส Mediatream

บ้นเคลของไดร์ฟเวอร์สเตอริโอจะพัฒนาโดยสืบทอดมาจากคลาส Device ของเฟรมเวิร์ก OSGI เพื่อให้เฟรมเวิร์กรู้ว่าภายในบ้นเคลนี้เป็นไดร์ฟเวอร์ของอุปกรณ์

Message ของไดร์ฟเวอร์สเตอริโอประกอบด้วย AudioPacketMessage และ BufferStatusMessage เมื่อเริ่มต้นการเล่นเพลงแล้วไบต์ถัดไปของข้อมูลเพลงจะถูกส่งไปยังอุปกรณ์สเตอริโอก็ต่อเมื่อได้รับการร้องขอมาจากสเตอริโอเท่านั้น

เมื่อข้อมูลในบัฟเฟอร์ของสเตอริโอใกล้จะหมด สเตอริโอจะส่ง BufferStatusMessage ซึ่งมีสถานะ “ALMOST_EMPTY” มายังไดร์ฟเวอร์สเตอริโอเพื่อขอข้อมูลเพิ่ม และหากข้อมูลในบัฟเฟอร์ของสเตอริโอหมดแล้ว สเตอริโอจะส่ง BufferStatusMessage ซึ่งมีสถานะ “EMPTY” มายังไดร์ฟเวอร์สเตอริโอเพื่อแสดงว่าเล่นจบเพลงแล้ว

AudioPacketMessage จะใช้ในการส่งข้อมูลเพลงไปให้สเตอริโอเมื่อมีการร้องขอมา โดยเมื่อไดร์ฟเวอร์สเตอริโอได้รับ AudioInputStream จากเซอร์วิส MediaStream แล้วก็จะจัดข้อมูลให้อยู่ในรูปแบบ AudioPacketMessage และส่งไปให้สเตอริโอผ่าน MBus

4.1.6.2 การจำลองอุปกรณ์สเตอริโอ

สเตอริโอที่พัฒนาขึ้นนี้เป็นจาวาแอปพลิเคชันที่มีหน้าที่เล่นเพลงซึ่งมีรูปแบบข้อมูลแบบ PCM และทำการเชื่อมต่อกับพอร์ต 4413 ของ MBus

สเตอริโอจะอ่าน AudioPacketMessage ที่ได้รับจากไดร์ฟเวอร์สเตอริโอและเล่นเพลงโดยเรียกใช้แพ็คเกจ javax.sound.sampled ในจาวา และส่ง BufferStatusMessage กลับไปให้ไดร์ฟเวอร์ โดยส่ง message ALMOST_EMPTY กลับไปให้ไดร์ฟเวอร์เพื่อขอข้อมูลเพิ่ม และส่ง message EMPTY กลับไปให้ไดร์ฟเวอร์เมื่อเล่นเพลงจบหรือเกิดเหตุการณ์ “buffer under-run”